

AIE683 Machine Learning

Week 6: Neural Networks (MLP)

Dr. Hakan Emekci

Hacettepe University, Graduate School of Informatics

A question you will be asked at work

Your team predicts **house prices** (or credit risk, churn, demand) from a table: 20 000 rows, 8–50 numeric and categorical columns.

A manager asks:

"Deep learning won in images, speech and language. Shouldn't we use a neural network?"

Today we answer this with evidence, not opinion:

1. What an MLP computes
2. How it is trained, and what makes training fail
3. How to regularize and tune it
4. Code: scikit-learn and PyTorch
5. **Head-to-head on real data:** MLP vs gradient boosting (Week 5)

Plan for today (3 hours)

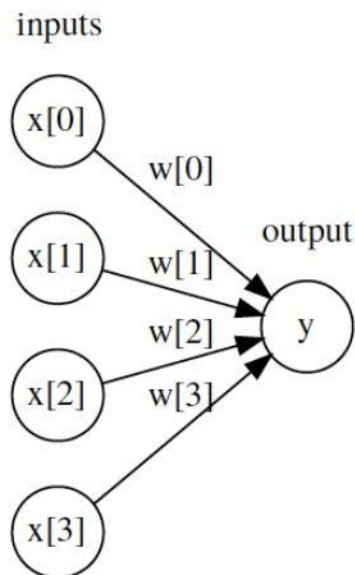
Block	Content	Time
1	The model: neurons, layers, activations, outputs	30 min
2	Training: backprop, SGD, momentum, Adam, initialization	40 min
3	Regularization: weight decay, early stopping, dropout, scaling	25 min
4	MLPClassifier / MLPRegressor + Try it 1	35 min
5	A PyTorch training loop + MLflow + Try it 2	35 min
6	MLP vs gradient boosting + Try it 3	25 min

Notebook: `lab.ipynb` in this week's folder.

History, briefly

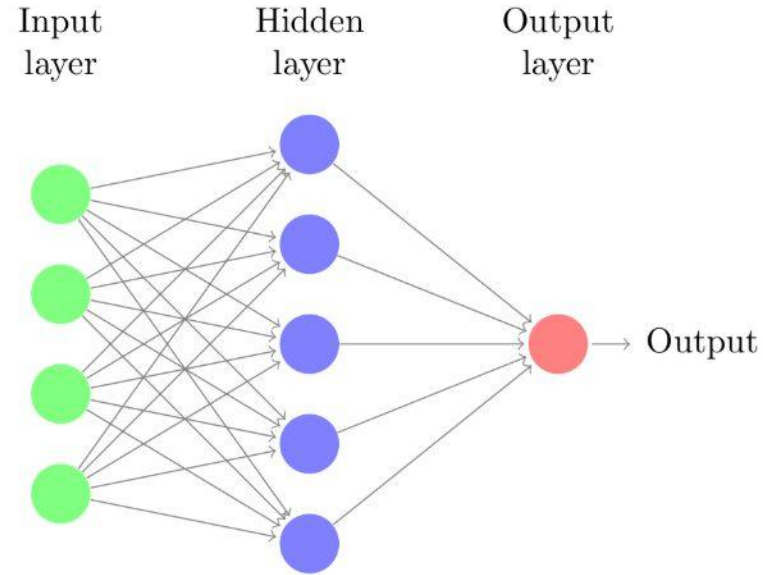
- Most ingredients of today's lecture existed by ~1990: the perceptron (1958), backpropagation (popularized 1986), universal approximation (1989).
- What changed after ~2010:
 - much more data
 - GPUs and software with automatic differentiation (PyTorch, JAX)
 - better defaults: ReLU, good initialization, dropout, Adam, normalization layers
- The big wins came on **images, audio and text**.
- On **tabular data**, gradient-boosted trees remain the strong default (we will see why).

Logistic regression as a neural net



$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{x} + b)$$

One hidden layer



$$h(x) = f(W_1x + b_1)$$

$$o(x) = g(W_2h(x) + b_2)$$

Why the nonlinearity is essential

Without f :

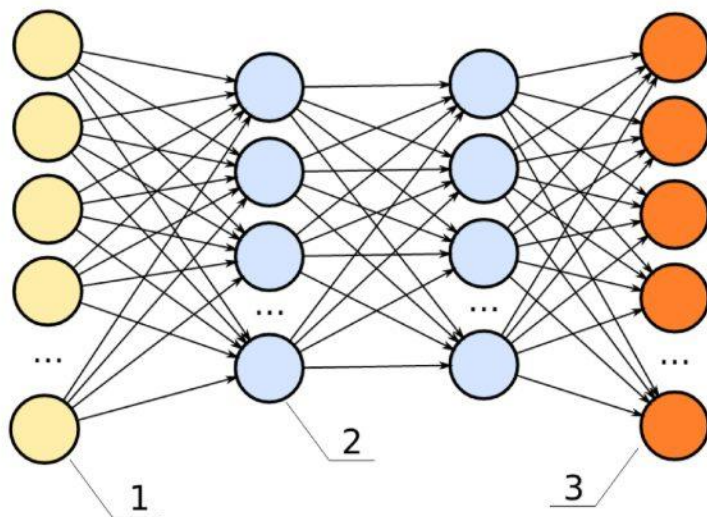
$$o(x) = W_2(W_1x + b_1) + b_2 = \underbrace{(W_2W_1)}_W x + \underbrace{(W_2b_1 + b_2)}_b$$

→ still a **linear model**, however many layers.

With f :

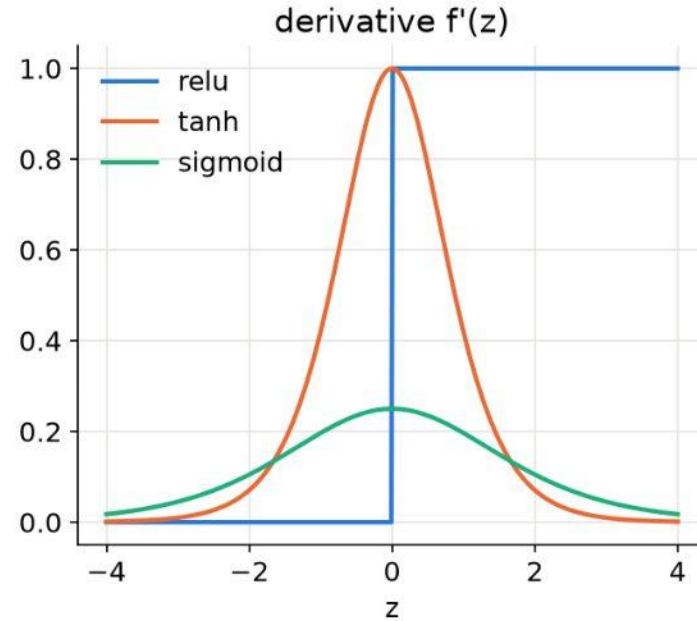
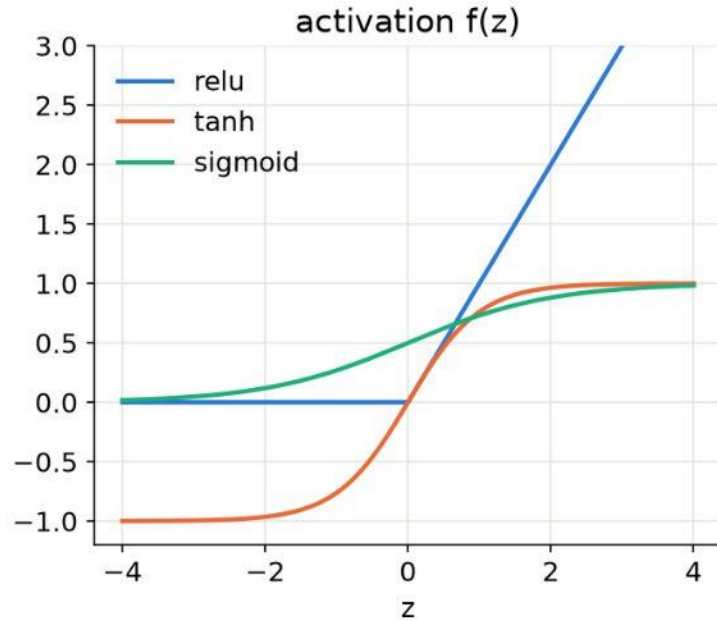
- each hidden unit is a learned nonlinear feature (a "bent" hyperplane for ReLU)
- one hidden layer with enough units can approximate any continuous function on a bounded domain (**universal approximation**)
- the theorem says nothing about how many units, how much data, or whether training finds the weights

Deeper networks



- Hidden layers usually share the same activation f ; each layer has its own weights.
- Names: multilayer perceptron (MLP), fully connected / dense / feed-forward network.
- "Deep learning" = many hidden layers.

Activation functions



Output layer and loss follow the task

Task	Output units	Output g	Loss
Regression	1 (per target)	identity	squared error
Binary classification	1	sigmoid	binary cross-entropy (log loss)
Multiclass (k classes)	k	softmax	cross-entropy

- Same choices as linear models in Week 4: an MLP is a GLM on learned features.
- In PyTorch, pass **logits** to `BCEWithLogitsLoss` / `CrossEntropyLoss`; they apply sigmoid/softmax internally (numerically stable).

Counting parameters

Network $8 \rightarrow 128 \rightarrow 128 \rightarrow 1$ (California housing):

Layer	Weights	Biases	Total
$8 \rightarrow 128$	$8 \cdot 128 = 1\,024$	128	1 152
$128 \rightarrow 128$	16 384	128	16 512
$128 \rightarrow 1$	128	1	129
Total			17 793

- A layer from n_{in} to n_{out} units has $(n_{\text{in}} + 1) n_{\text{out}}$ parameters.
- Width grows the count quadratically; the first layer grows with the number of features (one-hot encoding!).

Training objective

$$\min_{W_1, b_1, W_2, b_2} \frac{1}{N} \sum_{i=1}^N \ell(y_i, g(W_2 f(W_1 x_i + b_1) + b_2)) + \lambda \sum_1 \|W_1\|_2^2$$

- Same empirical risk minimization as in Week 4, with a more complex model.
- **Not convex**: many local minima and saddle points; the result depends on initialization, optimizer, learning rate, batch size.
- We are satisfied with a "good" solution, judged on **validation data**.

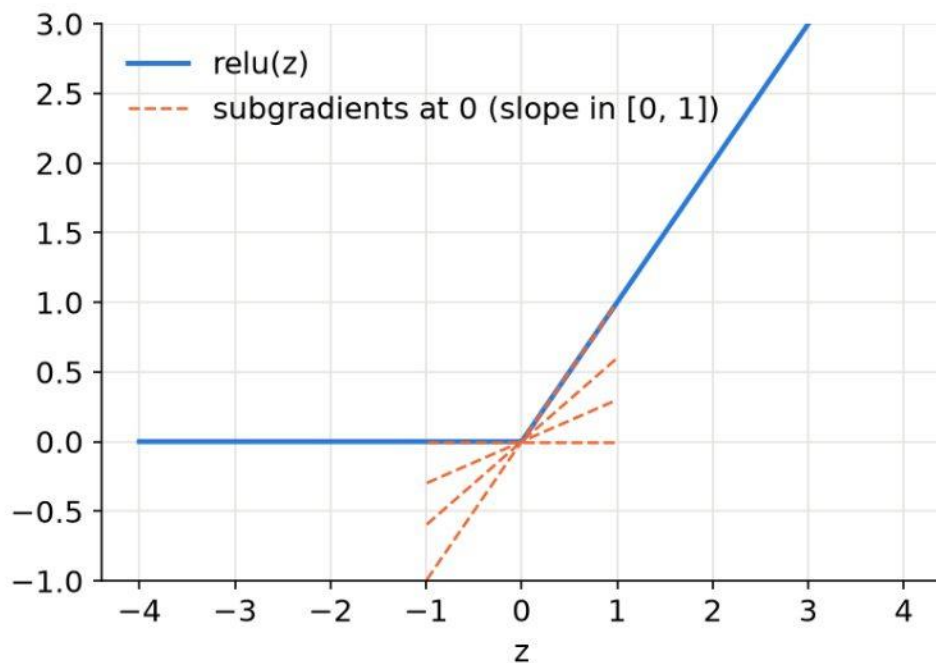
Backpropagation = chain rule, organized

Need $\frac{\partial \ell}{\partial W_1}$ for every layer. With $\text{net}(\mathbf{x}) = W_1 \mathbf{x} + b_1$:

$$\frac{\partial o(\mathbf{x})}{\partial W_1} = \underbrace{\frac{\partial o(\mathbf{x})}{\partial h(\mathbf{x})}}_{\text{backpropagation of gradient of layer above.}} \underbrace{\frac{\partial h(\mathbf{x})}{\partial \text{net}(\mathbf{x})}}_{\text{Gradient of Non-linearity } f} \underbrace{\frac{\partial \text{net}(\mathbf{x})}{\partial W_1}}_{\text{Input to 1st layer } \mathbf{x}}$$

- **Forward pass:** compute and store every layer's activations.
- **Backward pass:** from the loss back to the input, multiply local derivatives.
- Cost of the backward pass $\approx 2 \times$ forward pass. It computes gradients; it is **not** an optimizer.

But ReLU is not differentiable at 0



- Use a **subgradient**: any slope in $[0, 1]$ at $z = 0$; libraries pick 0.
- In floating point, exactly 0 almost never occurs.

Automatic differentiation in PyTorch

```
import torch
w = torch.tensor([1.0, -2.0], requires_grad=True)
x = torch.tensor([3.0, 4.0])
loss = torch.relu(w @ x + 1.0) ** 2    # forward pass records a graph
loss.backward()                        # backward pass: reverse-mode
print(loss.item(), w.grad)
```

0.0 tensor([0., 0.])

- Every operation records how to compute its local derivative.
- `backward()` walks the recorded graph from the loss to the parameters.
- You write the forward pass only; gradients come for free.

Gradient descent variants

Batch:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{1}{N} \sum_{i=1}^N \nabla_{\mathbf{W}} \ell(\mathbf{x}_i, y_i)$$

Stochastic (one sample):

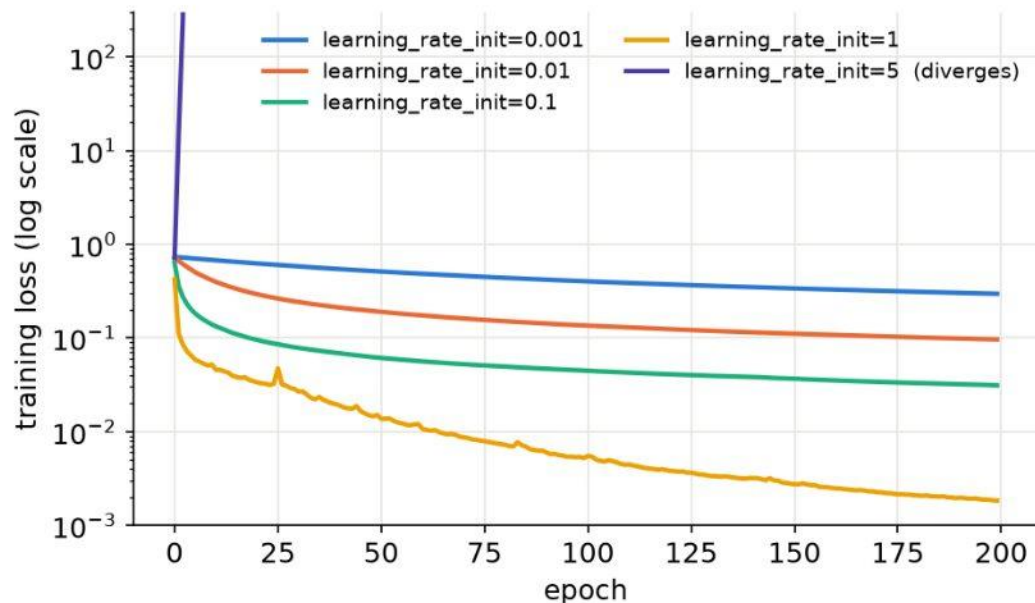
$$\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} \ell(\mathbf{x}_i, y_i)$$

Mini-batch (B samples, typically 32–512):

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{1}{|B|} \sum_{i \in B} \nabla_{\mathbf{W}} \ell(\mathbf{x}_i, y_i)$$

One pass over the training data = one **epoch**.

The learning rate matters most



`MLPClassifier(solver="sgd", momentum=0)` on breast cancer, scaled inputs.

Momentum

$$\mathbf{v} \leftarrow \beta \mathbf{v} + \nabla_{\mathbf{W}} \ell, \quad \mathbf{W} \leftarrow \mathbf{W} - \eta \mathbf{v} \quad (\beta \approx 0.9)$$

- Keeps a running direction: damps oscillation across steep directions, accelerates along shallow ones.
- Effective step size $\approx \eta/(1 - \beta)$: lower η when adding momentum.
- Nesterov momentum: evaluate the gradient at the look-ahead point; usually slightly better.

Adaptive learning rates: Adam

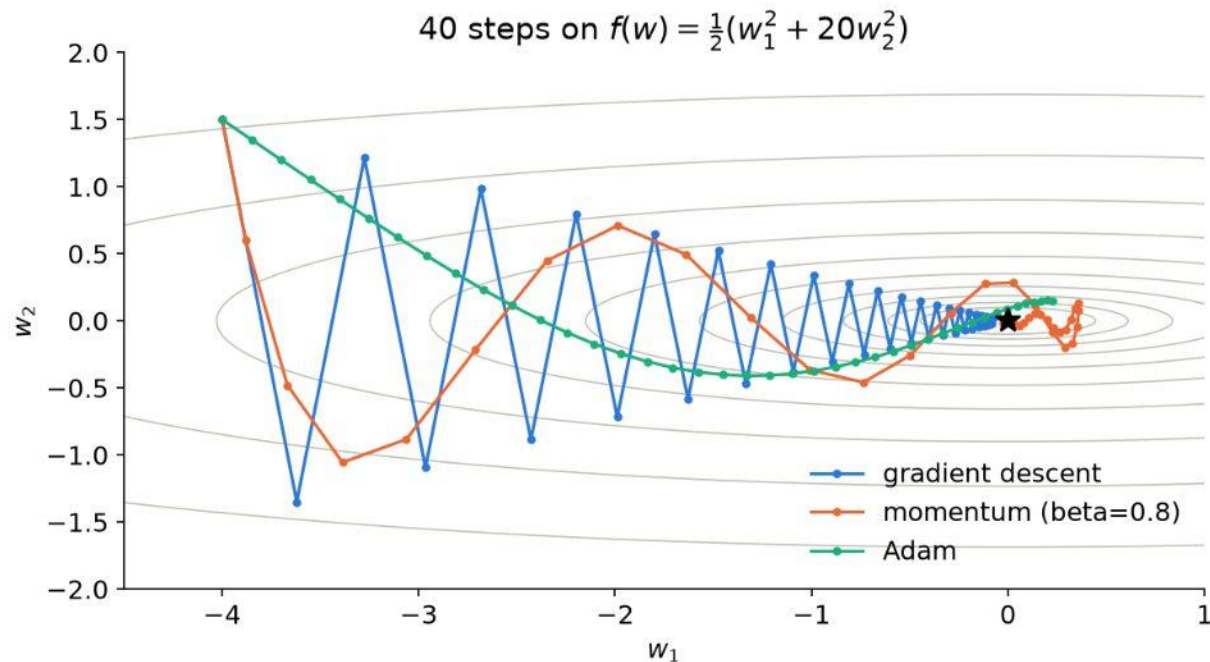
$$m \leftarrow \beta_1 m + (1 - \beta_1)g, \quad s \leftarrow \beta_2 s + (1 - \beta_2)g^2$$

$$W \leftarrow W - \eta \frac{\hat{m}}{\sqrt{\hat{s} + \epsilon}} \quad (\hat{m}, \hat{s} \text{ bias-corrected})$$

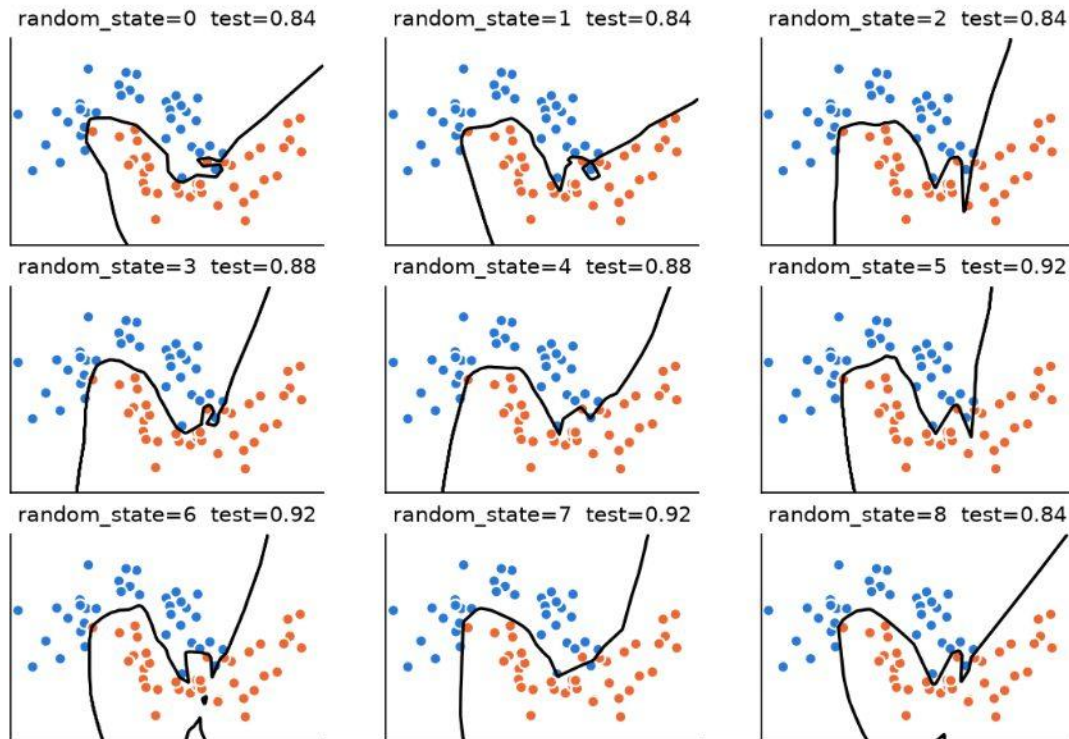
- Per-parameter step sizes: large for rarely/weakly updated weights, small for noisy ones.
- Defaults $\beta_1 = 0.9, \beta_2 = 0.999, \eta = 10^{-3}$ work surprisingly often.
- Relatives: AdaGrad, RMSProp. Use **AdamW** (decoupled weight decay) when you regularize.

Kingma & Ba, *Adam*, ICLR 2015 · Loshchilov & Hutter, *Decoupled Weight Decay Regularization*, ICLR 2019

Optimizers on an ill-conditioned problem



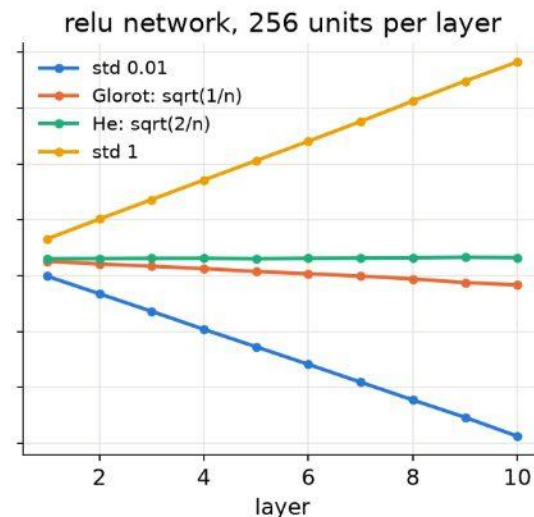
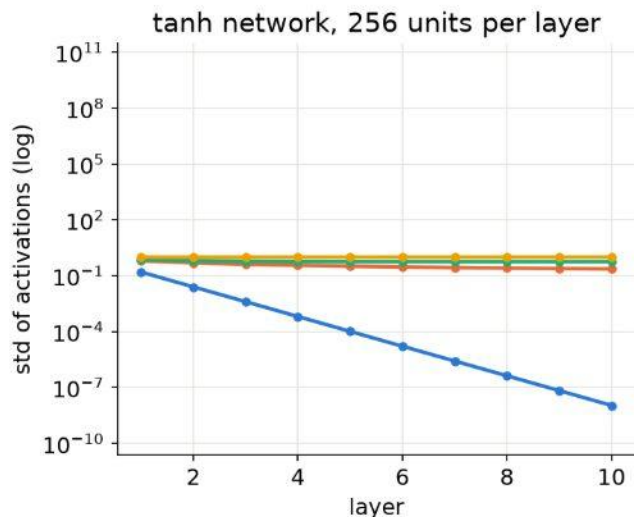
Non-convexity: results depend on the seed



`MLPClassifier(solver="lbfgs")` with 100 hidden units, 75 training points, 9 different `random_state` values.

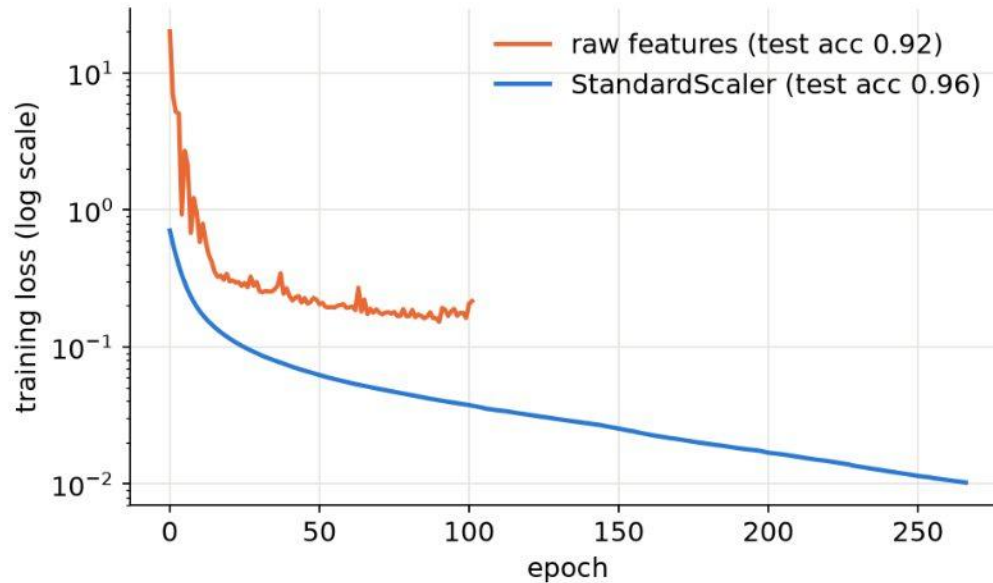
Initialization

- All zeros $\rightarrow$ every unit in a layer gets the same gradient forever (**symmetry**). Use random weights.
- The **scale** of the random weights controls whether signals and gradients vanish or explode:



Glorot/Xavier: $\text{Var}(\mathbf{W}) = 1/n_{\text{in}}$ (tanh, variants use $2/(n_{\text{in}} + n_{\text{out}})$) · He/Kaiming: $2/n_{\text{in}}$ (ReLU)

Scale your inputs



- Same `MLPClassifier` (Adam, defaults), breast cancer data, raw vs `StandardScaler`.
- Fit the scaler on training data only: put it in a `Pipeline` (Week 3).

Regularization

Complexity control for MLPs

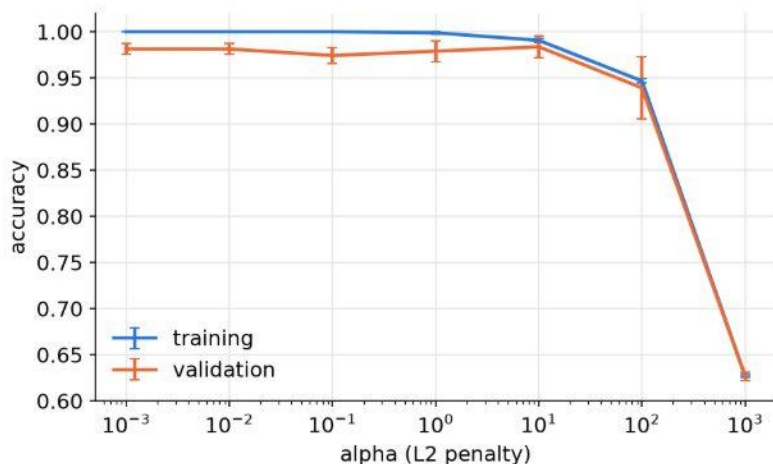
Knob	Effect	scikit-learn	PyTorch
Size (width, depth)	fewer parameters	<code>hidden_layer_sizes</code>	layer sizes
L2 / weight decay	shrink weights	<code>alpha</code>	<code>weight_decay</code> in AdamW
Early stopping	stop before overfitting	<code>early_stopping=True</code>	your loop
Dropout	noise on hidden units	—	<code>nn.Dropout(p)</code>

A common recipe: a network somewhat **too large**, then regularize and early-stop.

L2 penalty (weight decay)

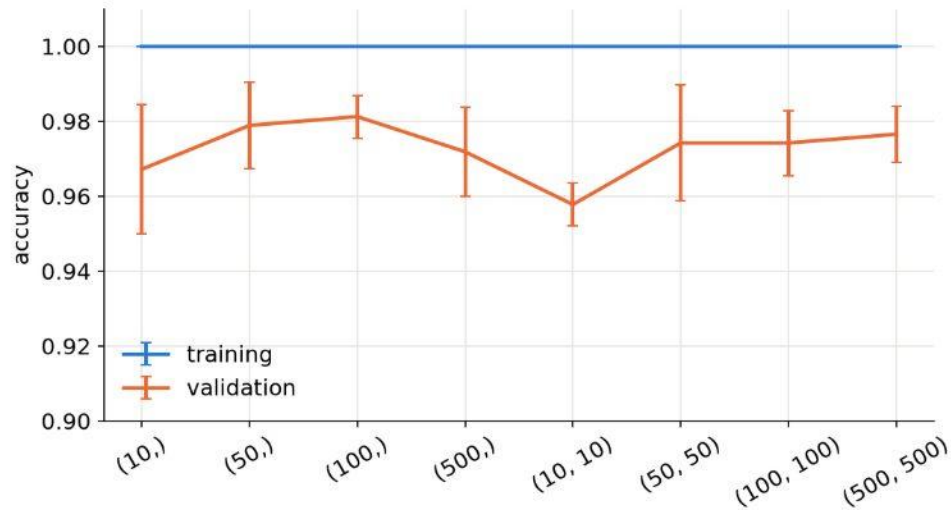
$$\min_{\mathbf{W}} \frac{1}{N} \sum_i \ell(y_i, o(x_i)) + \alpha \sum_l \|\mathbf{W}_l\|_2^2$$

```
pipe = make_pipeline(StandardScaler(),  
                    MLPClassifier(solver="lbfgs", max_iter=2000, random_state=1))  
param_grid = {"mlpclassifier__alpha": np.logspace(-3, 3, 7)}  
grid = GridSearchCV(pipe, param_grid, return_train_score=True).fit(X_train, y_train)
```

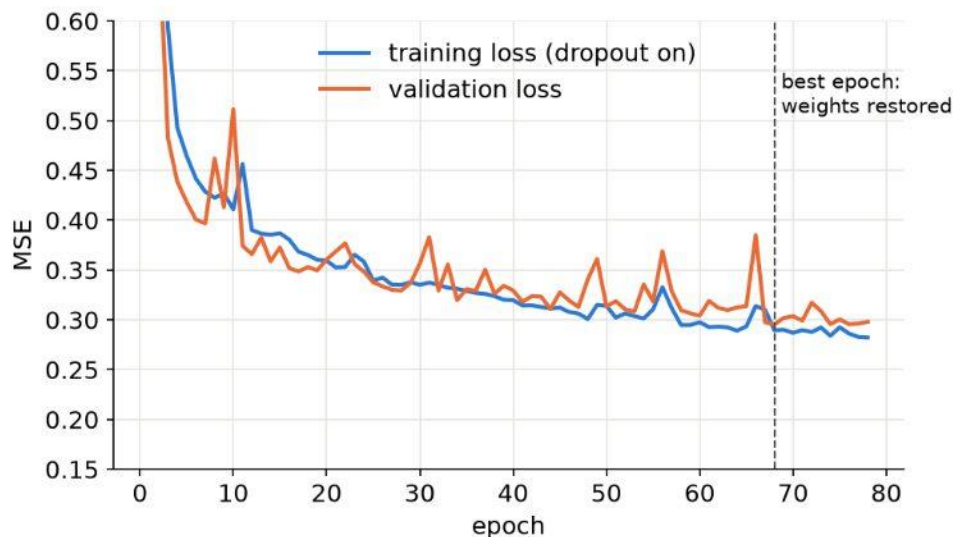


Network size

```
param_grid = {"mlpclassifier_hidden_layer_sizes":  
              [(10,), (50,), (100,), (500,), (10, 10), (50, 50), (100, 100), (500,  
grid = GridSearchCV(pipe, param_grid, return_train_score=True).fit(X_train, y_train
```



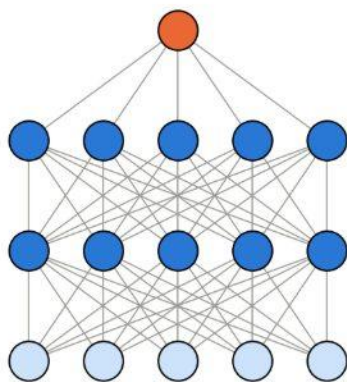
Early stopping



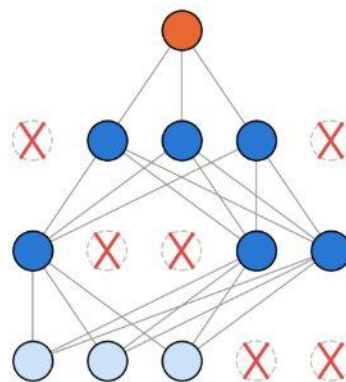
- Monitor the loss on a **validation set** after each epoch; stop when it has not improved for `patience` epochs; **restore the best weights**.
- scikit-learn: `early_stopping=True`, `validation_fraction=0.1`, `n_iter_no_change=10`.

Dropout

full network (used at prediction time)



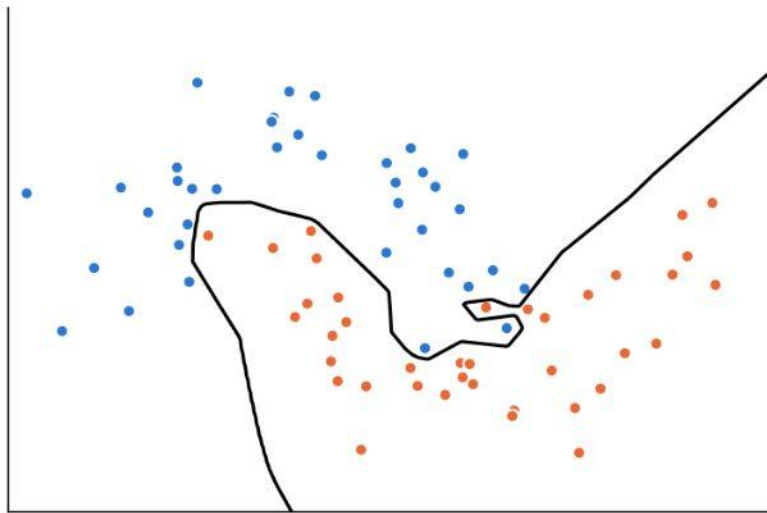
one training step with dropout



- In each training step, set each hidden unit to 0 with probability p (0.1–0.5); rescale survivors by $1/(1 - p)$.
- At prediction time: use all units, no noise (`model.eval()`).
- Interpretation: training an ensemble of thinned networks that share weights.

MLPs in scikit-learn

MLPClassifier on two moons



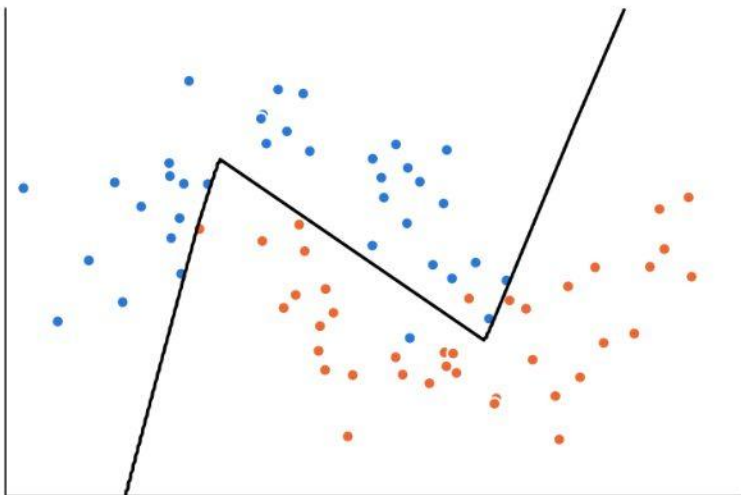
```
mlp = MLPClassifier(solver="lbfgs", max_iter=1000, random_state=0).fit(X_train, y_train)
print(mlp.score(X_train, y_train), mlp.score(X_test, y_test))
```

1.0 0.84

Hidden layer size

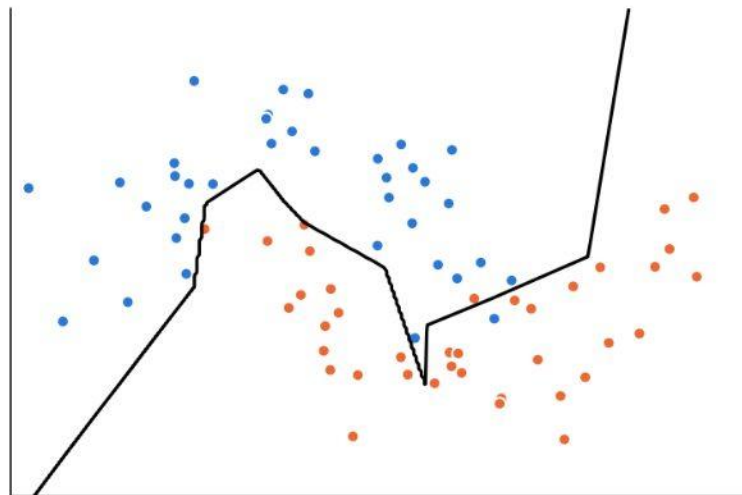
```
mlp = MLPClassifier(solver="lbfgs",  
    hidden_layer_sizes=(5,),  
    max_iter=1000, random_state=0)
```

train 0.97 test 0.84

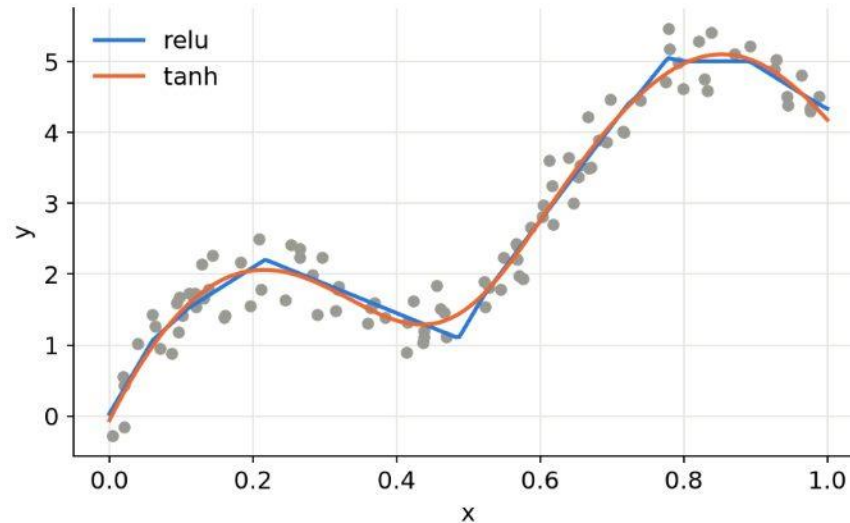


```
mlp = MLPClassifier(solver="lbfgs",  
    hidden_layer_sizes=(10, 10, 10),  
    max_iter=1000, random_state=0)
```

train 0.97 test 0.84



MLPRegressor



```
mlp_relu = MLPRegressor(solver="lbfgs", max_iter=2000, random_state=0).fit(X, y)
mlp_tanh = MLPRegressor(solver="lbfgs", activation="tanh", max_iter=2000, random_st
```

Try it 1: can an MLP beat logistic regression on credit risk?

Data: German credit (`fetch_openml("credit-g")`), 1 000 applicants, 20 features (13 categorical). Target: good / bad credit.

Task (12 min, `lab.ipynb`, section *Try it 1*):

1. Build a pipeline: `ColumnTransformer` (one-hot + scaling) → `MLPClassifier`.
2. Compare with `LogisticRegression` using 5-fold CV and ROC AUC.
3. Tune `alpha` and `hidden_layer_sizes` with `GridSearchCV`. Can you beat logistic regression by more than one standard deviation?

Write down your prediction before you run anything.

Limits of scikit-learn's MLP

- CPU only, no GPU.
 - Fixed architecture: dense layers, one activation, no dropout, no normalization layers.
 - One loss per estimator; no custom losses, multiple inputs, or embeddings for categories.
 - Fine for small and medium tabular problems, teaching, and baselines.
- For anything more flexible: **PyTorch** (or JAX; Keras 3 is a high-level API over JAX, PyTorch or TensorFlow).

A PyTorch training loop

Data: tensors and a DataLoader

```
import torch
from torch import nn
from torch.utils.data import TensorDataset, DataLoader

X, y = fetch_california_housing(return_X_y=True)           # 20 640 districts, 8 fea
X_trval, X_test, y_trval, y_test = train_test_split(X, y, test_size=0.2, random_sta
X_train, X_val, y_train, y_val = train_test_split(X_trval, y_trval, test_size=0.2,

scaler = StandardScaler().fit(X_train)                      # fit on training data o
def to_t(X, y):
    return (torch.tensor(scaler.transform(X), dtype=torch.float32),
            torch.tensor(y, dtype=torch.float32).unsqueeze(1))
Xtr, ytr = to_t(X_train, y_train); Xva, yva = to_t(X_val, y_val); Xte, yte = to_t(X
loader = DataLoader(TensorDataset(Xtr, ytr), batch_size=256, shuffle=True)
```

Model and optimizer

```
def make_mlp(n_in, hidden=(128, 128), dropout=0.1):
    layers, d = [], n_in
    for h in hidden:
        layers += [nn.Linear(d, h), nn.ReLU(), nn.Dropout(dropout)]
        d = h
    layers.append(nn.Linear(d, 1)) # identity output for regression
    return nn.Sequential(*layers)

torch.manual_seed(0)
model = make_mlp(X.shape[1])
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
loss_fn = nn.MSELoss()
print(sum(p.numel() for p in model.parameters())) # 17 793
```

- `nn.Linear` uses a Kaiming-style uniform initialization by default.

The training loop

```
best_val, best_state, best_epoch, patience = float("inf"), None, 0, 10
for epoch in range(150):
    model.train()
    for xb, yb in loader:
        opt.zero_grad()
        loss = loss_fn(model(xb), yb)
        loss.backward()
        opt.step()
    model.eval()
    with torch.no_grad():
        val_loss = loss_fn(model(Xva), yva).item()
    if val_loss < best_val:
        best_val, best_state, best_epoch = val_loss, copy.deepcopy(model.state_dict()), best_epoch + 1
    elif epoch - best_epoch >= patience:
        break
model.load_state_dict(best_state)
```

dropout on

clear old gradients

forward pass

backward pass (autodiff)

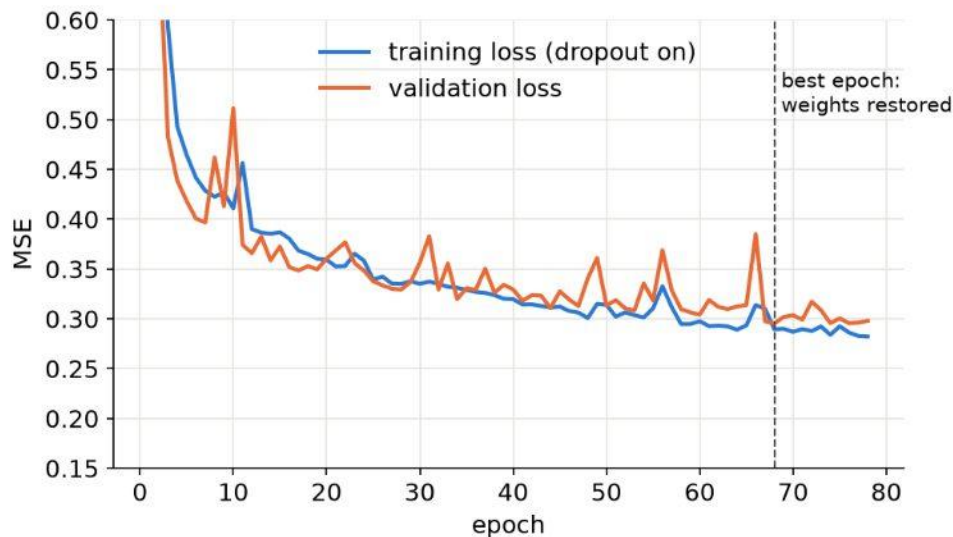
AdamW update

dropout off

early stopping

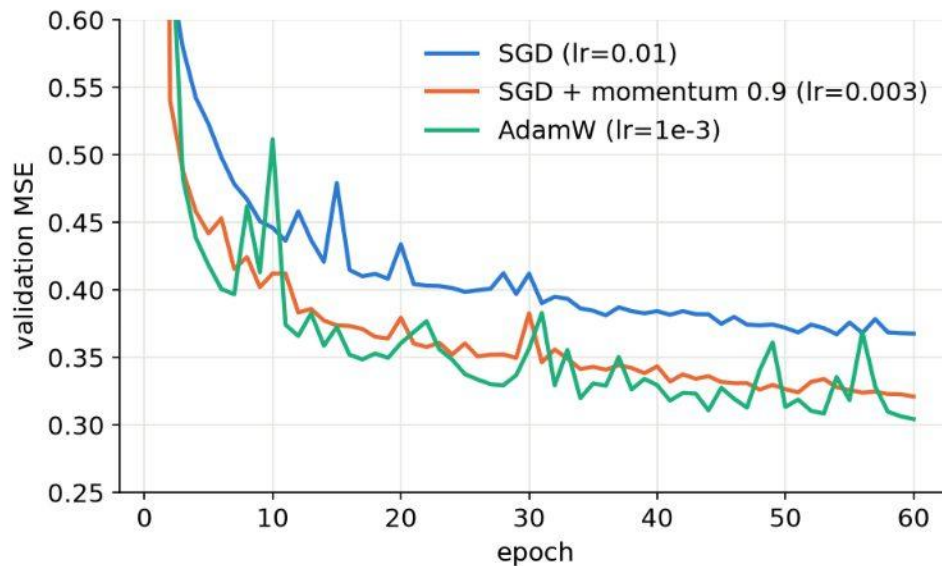
restore best weights

Result



Stopped after 78 epochs, best epoch 68, test $R^2 = 0.79$ (a few seconds on 2 CPU threads).

Comparing optimizers on the same network



- Same network, data and seed, 60 epochs each.
- Momentum needed a **smaller** learning rate (0.003); with 0.01 it diverged repeatedly.

Logging training runs to MLflow

```
import mlflow
mlflow.set_experiment("week06-neural-networks")

params = {"hidden": "128-128", "dropout": 0.1, "lr": 1e-3, "weight_decay": 1e-4,
          "batch_size": 256, "optimizer": "AdamW", "patience": 10}
with mlflow.start_run(run_name="torch-mlp"):
    mlflow.log_params(params)
    for epoch in range(len(hist["val"])):
        mlflow.log_metrics({"train_loss": hist["train"][epoch],
                           "val_loss": hist["val"][epoch]}, step=epoch)
    mlflow.log_metrics({"best_epoch": best_epoch, "test_r2": test_r2})
```

- `step=` turns metrics into **curves** in the MLflow UI; compare runs side by side.
- `uv run mlflow ui` in the week folder, then open <http://127.0.0.1:5000>.

What goes wrong in practice

Symptom	Likely cause
Loss is <code>nan</code> or explodes	learning rate too high; unscaled inputs or targets
Loss barely moves	learning rate too low; dead ReLUs; forgot <code>opt.step()</code>
Training loss decreases then jumps up	learning rate too high; no schedule
Great validation, poor test	tuned/early-stopped on the test set; leakage (scaler fitted on all data)
Predictions change between calls	forgot <code>model.eval()</code> (dropout active)
Loss is strangely noisy/large	forgot <code>opt.zero_grad()</code> (gradients accumulate)
Regression loss plateaus high	target shape <code>(n,)</code> vs output <code>(n, 1)</code> broadcast in <code>MSELoss</code>

Try it 2: find the bugs

The lab contains a training loop, `buggy_train()`, for the California housing MLP. It **runs without errors** but trains badly.

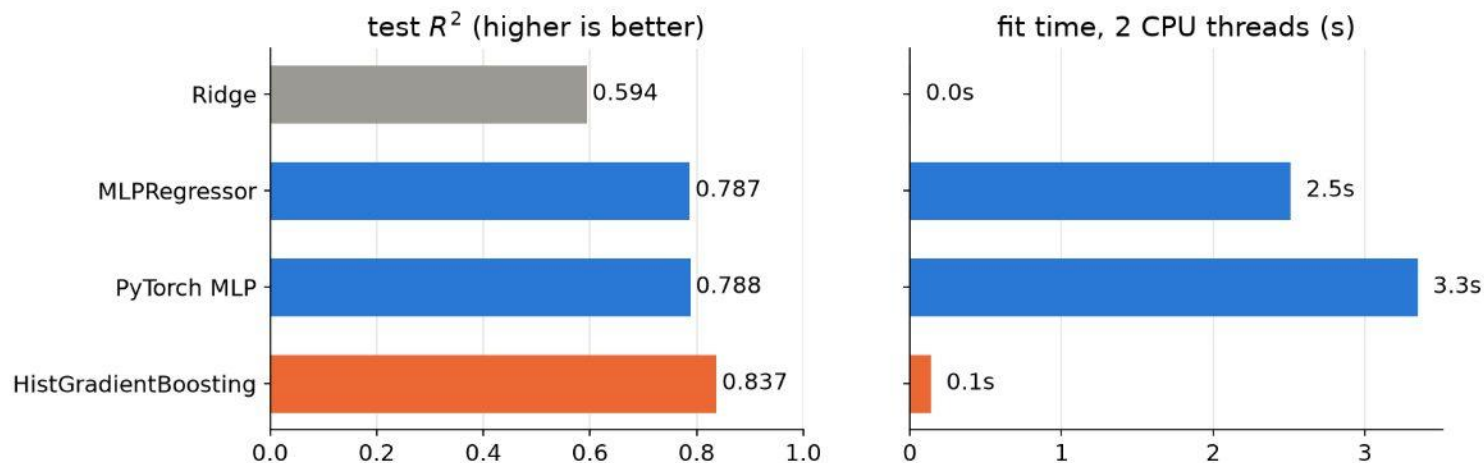
Task (12 min, `lab.ipynb`, section *Try it 2*):

1. Run it and look at the training and validation curves. What looks wrong?
2. Find the **three** planted bugs (use the table on the previous slide).
3. Fix them one at a time and log each version as an MLflow run, so you can compare the curves in the UI.

Target: validation MSE below 0.35.

MLP vs gradient boosting on tabular data

California housing: the numbers



Same train/test split; default `HistGradientBoostingRegressor`, lightly tuned MLPs.

What the benchmarks say

- Large benchmarks on medium-sized tabular datasets (~10k rows): tree ensembles (XGBoost, LightGBM, CatBoost, HistGradientBoosting) win most often, with far less tuning.
- Why trees fit tabular data well:
 - robust to uninformative features and to feature scales
 - target functions are often irregular / non-smooth; MLPs are biased towards smooth functions
 - features have individual meaning; an MLP mixes them (rotation invariance)
- Well-tuned and regularized MLPs can close much of the gap, at a much higher tuning cost.

Grinsztajn, Oyallon & Varoquaux, *Why do tree-based models still outperform deep learning on typical tabular data?*, NeurIPS

2022 Datasets & Benchmarks · Shwartz-Ziv & Armon, *Tabular data: Deep learning is not all you need*, Information Fusion 2022 ·

Kadra et al., *Well-tuned simple nets excel on tabular datasets*, NeurIPS 2021

When an MLP is a reasonable choice on tables

- Very large datasets (hundreds of thousands of rows and more), mostly smooth numeric features
- The model must be trained end-to-end with other inputs (text, images) or share representations across tasks
- Multi-output or custom losses, differentiable pipelines
- Online / incremental learning (`partial_fit`, streaming updates)
- **Ensembles**: averaging an MLP with a boosted model often beats both (errors are less correlated)

Default workflow: linear baseline → gradient boosting → (MLP if one of the above applies) → compare with proper CV.

Try it 3: predict the winner, then run it

Data: forest cover type (`fetch_covtype`), 54 cartographic features, 7 classes; we use a 30 000-row subsample (16 000 train, 4 000 validation, 10 000 test).

Task (12 min, `lab.ipynb`, section *Try it 3*):

1. Write your prediction in the notebook: MLP or `HistGradientBoostingClassifier`, and by how much (accuracy)?
2. Train both on the same split (templates provided; each fits in seconds).
3. Make one change to improve either model, judged on **validation** accuracy (at most 2 minutes of fitting). Evaluate on the test set once, at the end.

Summary

- An MLP = stacked affine maps + nonlinearities; output layer and loss as in GLMs.
- Training: backprop computes gradients; mini-batch SGD / momentum / **AdamW** update; initialization and **input scaling** decide whether training works at all.
- Regularize with size, weight decay, **early stopping**, dropout; tune on validation data; fix seeds.
- scikit-learn `MLPClassifier`/`MLPRegressor` for baselines; PyTorch for flexibility; log curves to MLflow with `step=`.
- On tabular data, gradient boosting is the stronger default; MLPs are a complement, not a replacement.

Next week

Week 7: Evaluation

- Metrics beyond accuracy: precision, recall, ROC and PR curves
- Calibration of predicted probabilities
- Imbalanced data and choosing decision thresholds

Before next week: finish the three *Try it* sections in `lab.ipynb`.

Questions?

Backup

Backprop in matrix form

For layer l with pre-activation $z_l = W_l a_{l-1} + b_l$ and activation $a_l = f(z_l)$:

$$\delta_L = \frac{\partial \ell}{\partial z_L}, \quad \delta_l = (W_{l+1}^T \delta_{l+1}) \odot f'(z_l)$$

$$\frac{\partial \ell}{\partial W_l} = \delta_l a_{l-1}^T, \quad \frac{\partial \ell}{\partial b_l} = \delta_l$$

- For softmax + cross-entropy (and sigmoid + log loss): $\delta_L = \hat{y} - y$.
- $f'(z)$ appears in every step: saturated activations ($f' \approx 0$) $\rightarrow$ vanishing gradients.

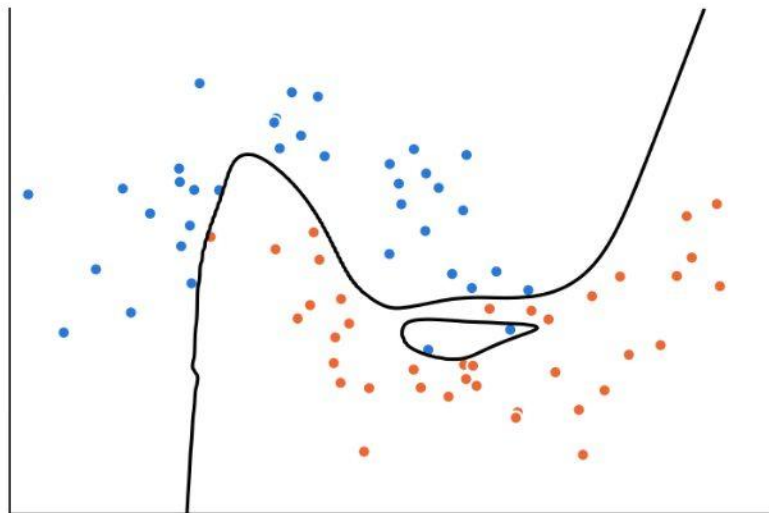
Learning-rate schedules and choosing an optimizer

- Schedules: step decay, **cosine decay**, **warm-up** then decay, "reduce on plateau".
- Rough guide:
 - small data, small net ($\approx 10k$ rows): L-BFGS (full batch) works well; `solver="lbfgs"` in scikit-learn
 - default: **AdamW**, $\eta \approx 10^{-3}$, maybe cosine decay
 - with time for tuning: SGD + momentum + schedule can match or beat Adam
- Batch size and learning rate interact: larger batches tolerate larger η .

Activation: tanh vs ReLU

```
mlp = MLPClassifier(solver="lbfgs",  
                    hidden_layer_sizes=(10, 10, 10),  
                    activation="tanh",  
                    max_iter=1000, random_state=0)
```

train 1.0 test 0.88



Key parameters and attributes

Parameter	Default	Notes
hidden_layer_sizes	(100,)	tuple: one entry per hidden layer
activation	"relu"	also "tanh", "logistic"
solver	"adam"	"lbfgs" for small data, "sgd" with momentum
alpha	1e-4	L2 penalty
learning_rate_init, batch_size	1e-3, min(200, n)	Adam/SGD only
early_stopping, n_iter_no_change	False, 10	holds out validation_fraction
max_iter	200	epochs for Adam/SGD

Attributes: `coefs_`, `intercepts_`, `loss_curve_`, `validation_scores_`, `n_iter_`.